Drone Delivery Management System

GROUP 7

Adheeb (16)
Kavindu (23)

INTRODUCTION

In this generation, delivery efficiency plays a major role in logistics and e-commerce industries. Traditional human delivery methods consume more time and cost, often causing delays due to traffic or long distances.

To overcome these challenges, drone-based delivery systems are emerging as a modern solution for automating small and medium package deliveries efficiently.

However, manually tracking multiple drones, their battery levels, capacities, and delivery statuses can be challenging for delivery companies.

The **Drone Delivery Management System** addresses this problem by providing a computerized, GUI-based Java application that allows company staff to manage drones and assign deliveries easily.

In this system, staff members can register drones by entering their **ID, capacity, and battery levels**. When a new package is added, the system **automatically selects the most suitable drone** based on its **availability, capacity, and battery level**, then simulates the delivery in real time. Once the delivery completes, the drone's status is updated, and the **total delivery price** is displayed based on **distance and weight**.

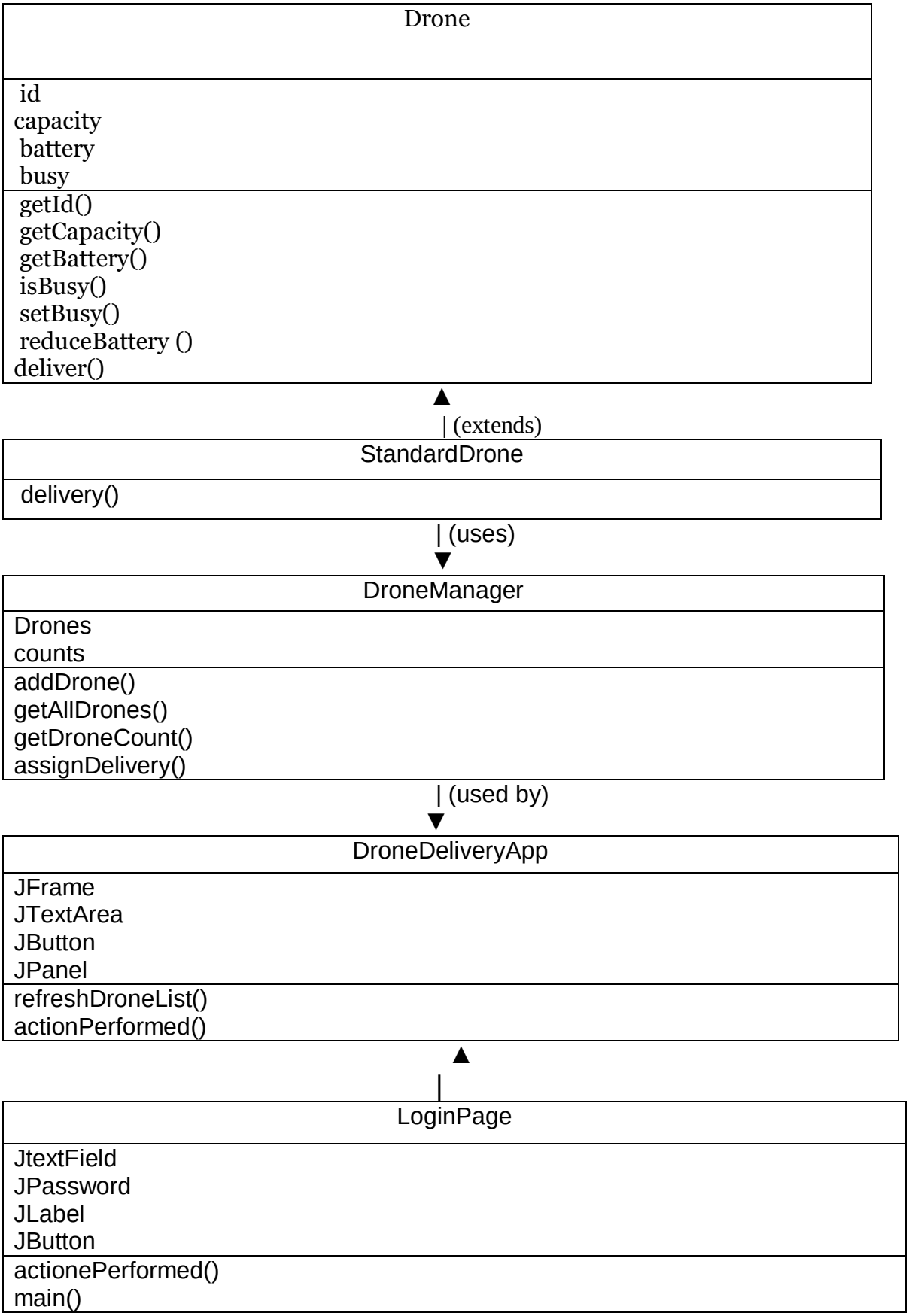
The main goals of the system are to:

- Manage drones easily through a GUI.
- Ensure efficient drone assignment based on capacity, distance, and battery.
- Simulate real-time delivery and battery consumption.
- Display delivery charge dynamically based on distance and weight.
- Allow only staff login access for security.
- Update the drone's battery level after each delivery.

System Features

- **Staff Login** – Only authorized staff (username: *admin*, password: *12345*) can access the system.
- **Drone Management** – Add new drones with unique ID, capacity, and battery level.
- **Package Assignment** – Dynamically assigns the most suitable drone.
- **Delivery Simulation** – Simulates delivery time using multithreading.
- **Battery & Time Formulae** – Calculates drone battery usage and delivery time based on weight and distance.
- **Pricing Calculation** – Adds VAT and delivery cost for each completed delivery.
- **Automatic Drone Status Update** – Battery and busy status update after delivery.
- **User-Friendly GUI** – Two panels (for drones and deliveries) and separate display areas.

UML DIAGRAM


Drone
id
capacity
battery
busy
getId()
getCapacity()
getBattery()
isBusy()
setBusy()
reduceBattery ()
deliver()
| (extends)
StandardDrone
delivery()
| (uses)
DroneManager
Drones
counts
addDrone()
getAllDrones()
getDroneCount()
assignDelivery()
| (used by)
DroneDeliveryApp
JFrame
JTextArea
JButton
JPanel
refreshDroneList()
actionPerformed()
LoginPage
JtextField
JPassword
JLabel
JButton
actionePerformed()
main()

Explanation of How OOP Concepts Are Applied

1. **Encapsulation —** Protecting data and controlling access

Where it is used:

```
private int id;
private double capacity;
private int battery;
protected boolean busy;

public int getId() { return id; }
public double getCapacity() { return capacity; }
public int getBattery() { return battery; }
public void setBusy(boolean busy) { this.busy = busy; }
```

Explanation:

- All the important variables are **declared private**, so they can't be accessed directly from outside the class.
- Accessing the variable is only possible through **public getter and setter methods**, which ensures **controlled modification** and prevents invalid data.
- For example, battery can only be reduced using the method reduceBattery(int amount), maintaining logical consistency.

Why we used:

- To protect drone data from being changed accidentally.
- To maintain system reliability by enforcing **data integrity**.
- To make debugging easier since all updates go through defined methods.

2. **Abstraction** — Hiding complex details, showing only essentials

Where it is used:

In the Drone class.

```
abstract class Drone {
   private int id;
   private double capacity;
   private int battery;
   protected boolean busy;

   public abstract String deliver(int packageId, double weight, double distance);
}
```

Explanation:

- The Drone class is a **blueprint** for all types of drones.
- It contains only the **essential Variables** and **common methods**.

Why it's useful:

- Simplifies the system by focusing on behavior rather than implementation details.
- Makes the code **easier to extend** — we can add more drone types later without changing existing logic.

3. **Inheritance —** Reusing and extending existing functionality

Where it is used:

StandardDrone class **inherits** from the Drone class.

```
class StandardDrone extends Drone {
   public StandardDrone(int id, double capacity, int battery) {
      super(id, capacity, battery);
   }

   public String deliver(int packageID, double weight, double distance) {
      // Specific delivery logic
   }
}
```

Explanation:

- The subclass StandardDrone automatically **inherits all Variables and Methods** from the Drone class.
- We can **add or override behaviors** that are specific to standard drones.
- Using super() keyword we can call the parent constructor to reuse setup logic.

Why it's useful:

- We can Reuse the code because common features are defined once in Drone.
- Makes the program **scalable** because new drone types can be added easily without rewriting existing code.

4. **Polymorphism —** One interface, multiple behaviors

Where it is used:

When assignDelivery() in **DroneManager** calls:

```
chosen.deliver(packageId, weight, distance);
```

the correct deliver() version runs automatically depending on whether the drone is StandardDrone or another type.
This is runtime polymorphism **(method overriding)**.

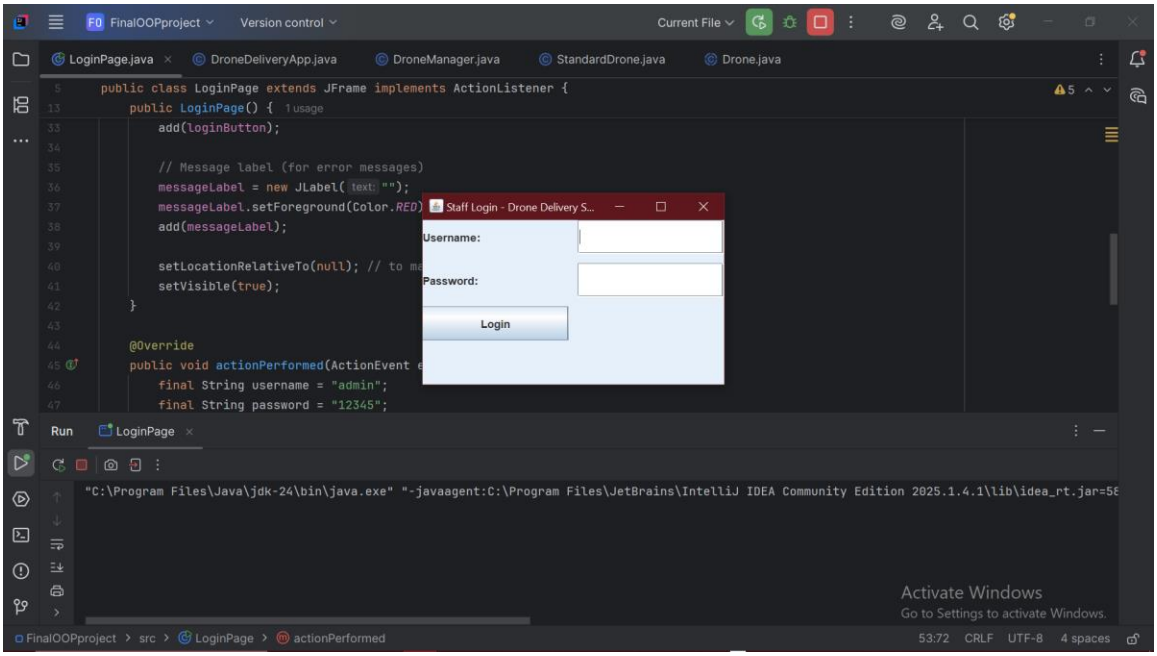
Explanation:

It allows the same method to perform differently based on the object type.
In our project, the abstract class **Drone** defines the abstract method **deliver()**, which is **overridden** in the subclass **StandardDrone** to give its own delivery behavior (battery use, time, and cost).

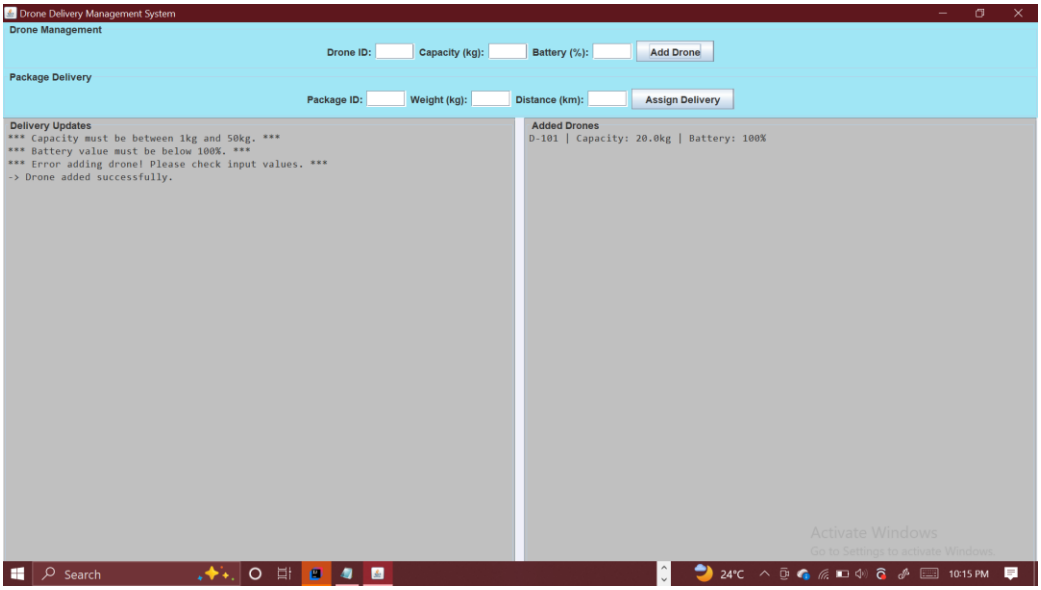
Why it's useful:

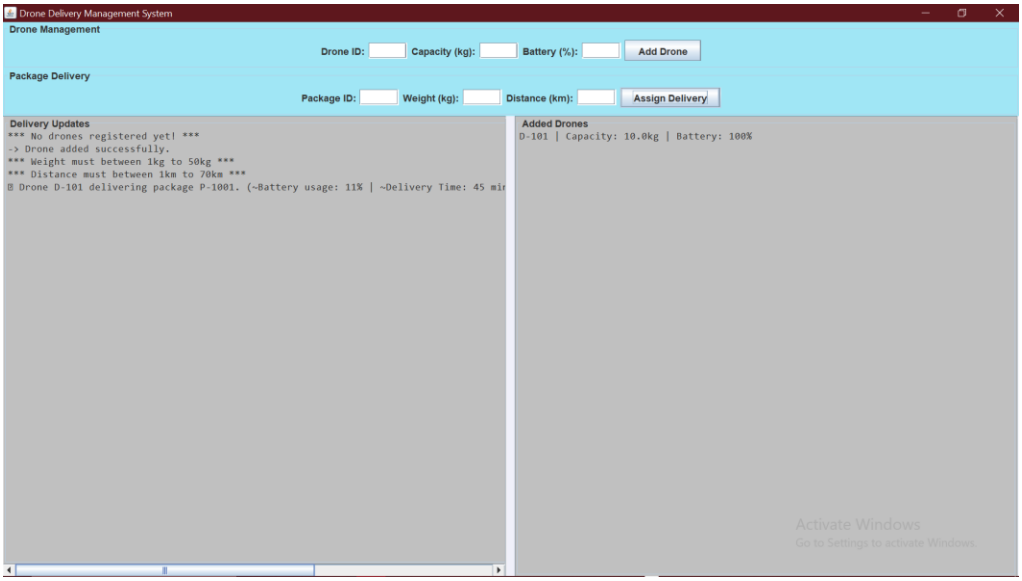
- Makes your system **flexible and dynamic**.
- New drone types (e.g., `PremiumDrone`) can be added without changing existing logic.

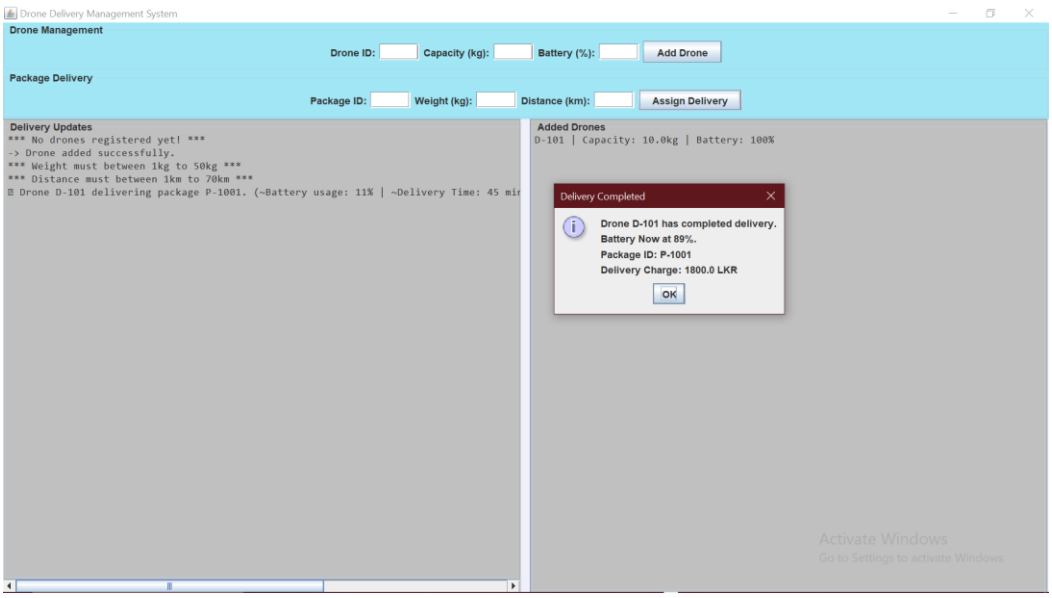
Screenshots of my GUI






Drone Delivery Management System
Drone Management
Drone ID:
Capacity (kg):
Battery (%):
Add Drone
Package Delivery
Package ID:
Weight (kg):
Distance (km):
Assign Delivery
Delivery Updates
*** Capacity must be between 1kg and 50kg. ***
*** Battery value must be below 100%. ***
*** Error adding drone! Please check input values. ***
-> Drone added successfully.
Added Drones
D-101 | Capacity: 20.0kg | Battery: 100%
Activate Windows
Go to Settings to activate Windows.
Search
24°C
10:15 PM


Drone Delivery Management System
Drone Management
Drone ID:
Capacity (kg):
Battery (%):
Add Drone
Package Delivery
Package ID:
Weight (kg):
Distance (km):
Assign Delivery
Delivery Updates
*** No drones registered yet! ***
-> Drone added successfully.
*** Weight must between 1kg to 50kg ***
*** Distance must between 1km to 70km ***
Drone D-101 delivering package P-1001. (~Battery usage: 11% | ~Delivery Time: 45 mi
Added Drones
D-101 | Capacity: 10.0kg | Battery: 100%
Activate Windows
Go to Settings to activate Windows.


Drone Delivery Management System
Drone Management
Drone ID:
Capacity (kg):
Battery (%):
Add Drone
Package Delivery
Package ID:
Weight (kg):
Distance (km):
Assign Delivery
Delivery Updates
*** No drones registered yet! ***
-> Drone added successfully.
*** Weight must between 1kg to 50kg ***
*** Distance must between 1km to 70km ***
Drone D-101 delivering package P-1001. (~Battery usage: 11% | ~Delivery Time: 45 mi
Added Drones
D-101 | Capacity: 10.0kg | Battery: 100%
Delivery Completed
Drone D-101 has completed delivery.
Battery Now at 89%.
Package ID: P-1001
Delivery Charge: 1800.0 LKR
OK
Activate Windows
Go to Settings to activate Windows.